Peer-to-peer File Sharing Web-Application

Prepared By: Shaw Campbell, Ella Neave, David Katz
Prepared For: CSC3002F
13/03/2025

Table of Contents

1. Features **2**
2. Protocol Specification **2**
3.1. Sequence Diagrams 3
3.2. State Diagrams 4
3.2. Message Formats (Shaw) 6
3.2.1. Leecher Messages 6
3.2.2. Seeder Messages 7
3.2.3. Tracker Messages 7
3. Screenshots **8**

1. Features

Table 1 shows the features of the system and a description of each.

Feature	**Description**
Register with Tracker as Leecher or Seeder	Leecher and seeder generate a random number as their user id and register with a tracker.
Request File	Leechers and seeders can download files from peers.
Parallel Download from Seeders	Client downloads files from multiple peers in parallel.
Optional Switch to Seeder after Download	Leechers can optionally switch to seeders after each download
Update on Seeder State	Seeders update tracker on the files it does and does not have every 5 seconds.
Checksum	Downloaded file is verified with a checksum

Table 1: Features

2. Protocol Specification

This section describes the protocol used by the web application with state and sequence diagrams.

3.1. Sequence Diagrams

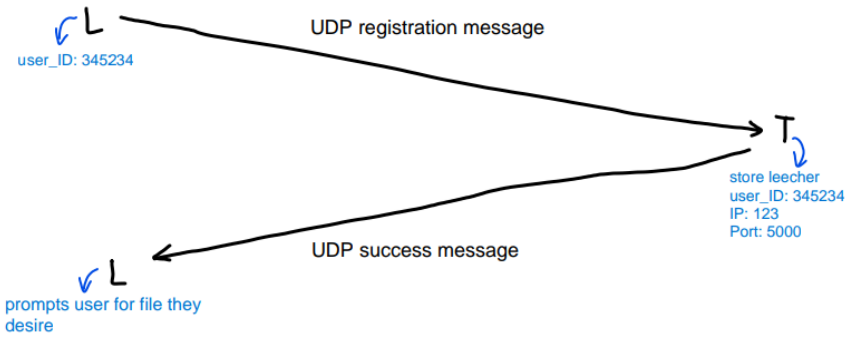


Figure 1: Leecher registering with tracker

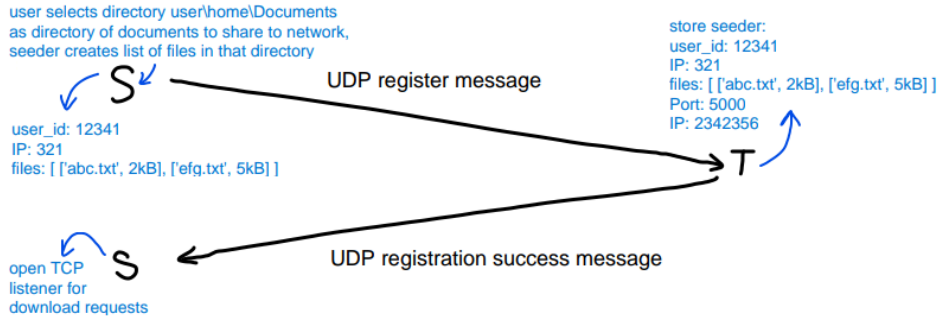
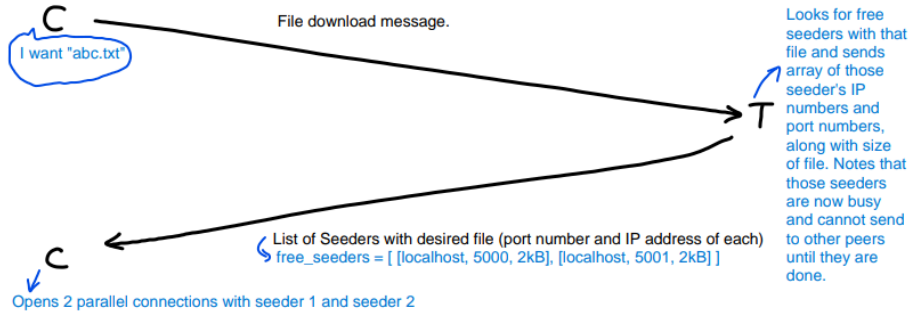
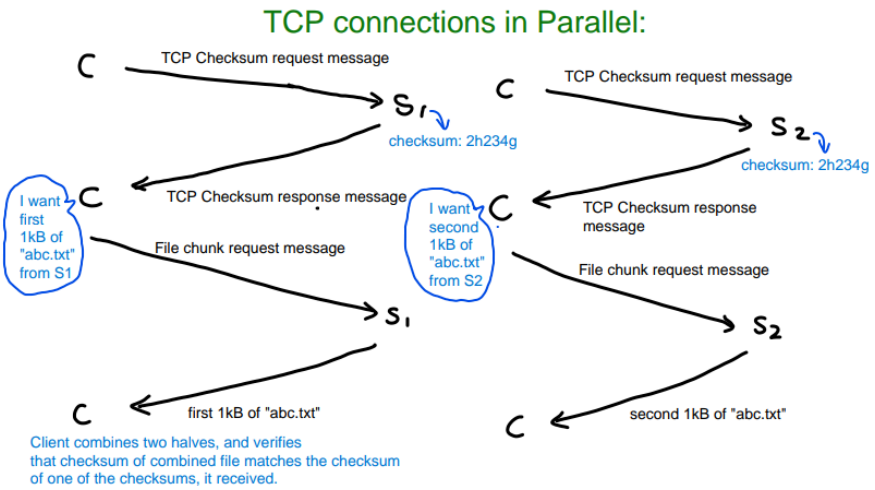


Figure 2: Seeder registering with tracker





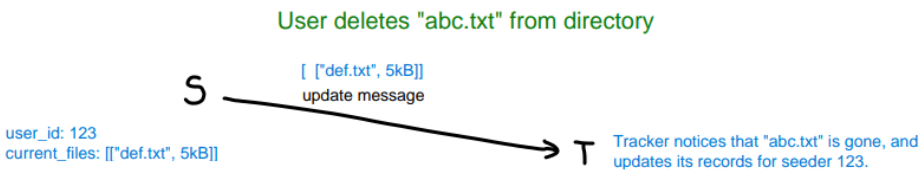


Figure 3: Client (leecher or seeder) downloading a file from network

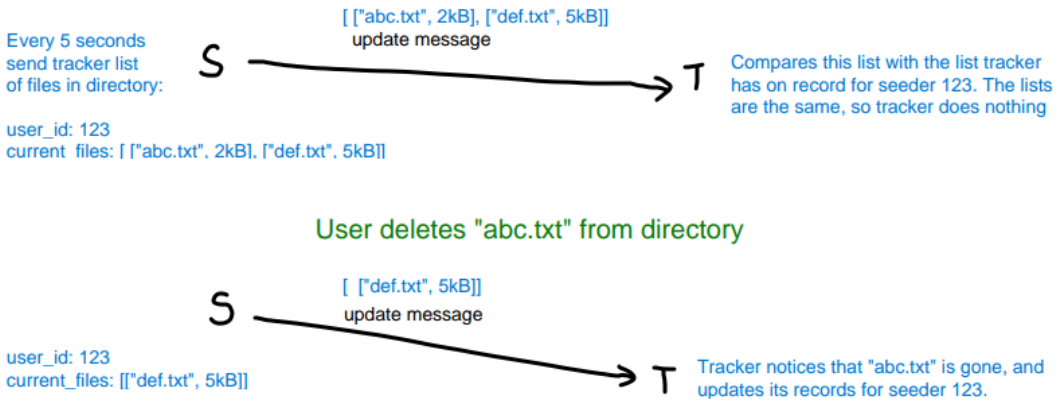
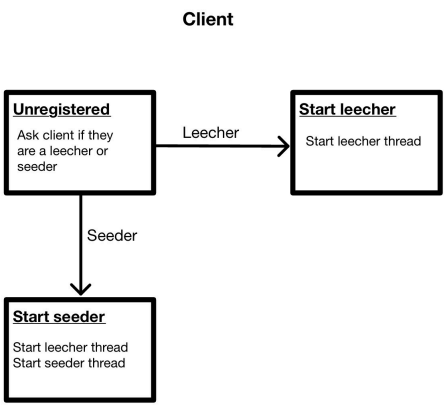
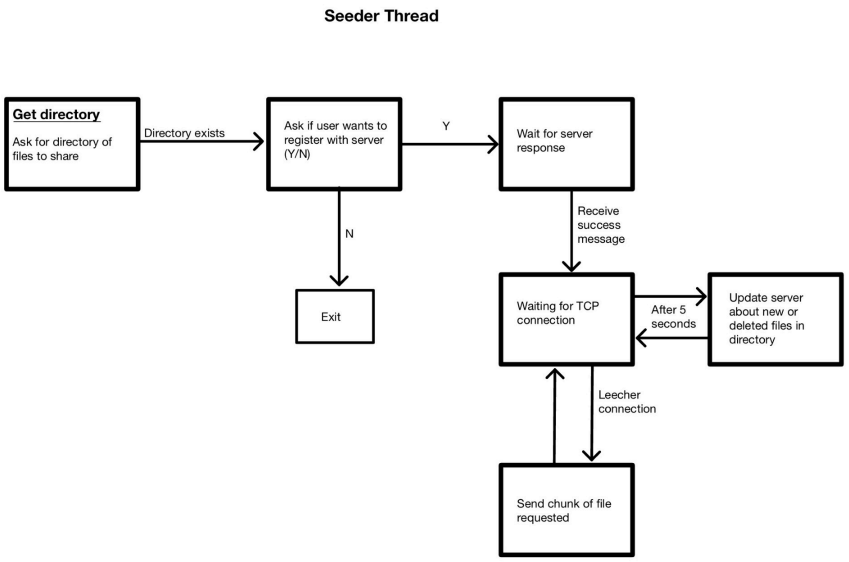


Figure 4: Seeder updating tracker about new or deleted files

3.2. State Diagrams





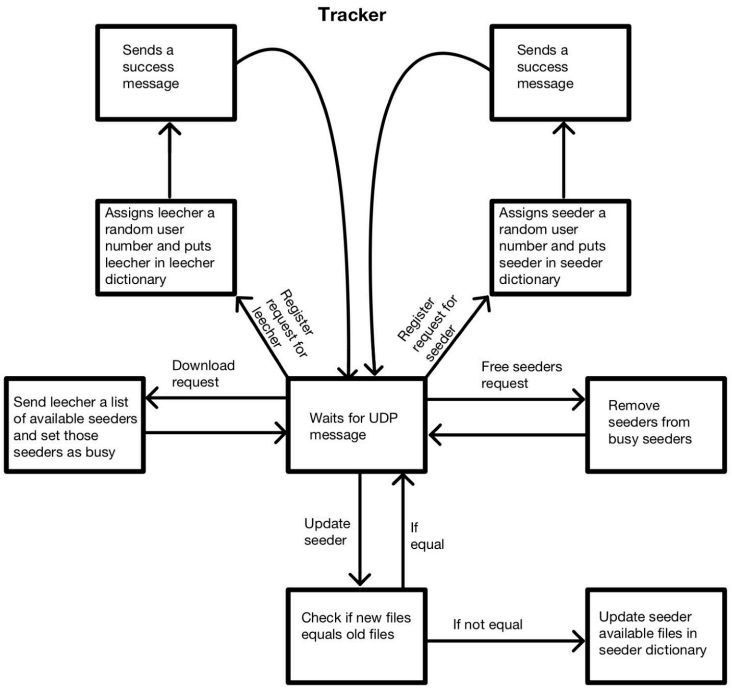


Figure 5: State flow diagrams for client, seeder thread and leecher thread.

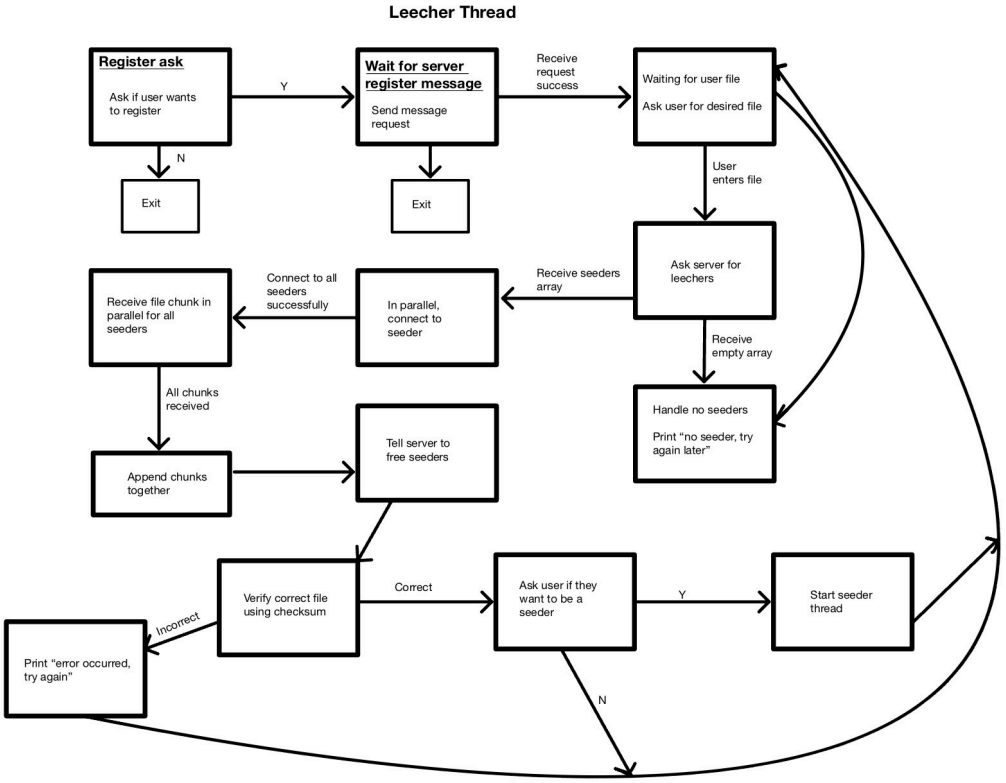


Figure 6: State diagram for tracker

3.2. Message Formats (Shaw)

For this project, JSON objects were used to encapsulate data. Therefore, no structure was required from one message to another, just the receivers prior knowledge of which field it will need. For instance, if the tracker requires the code from a message it has received, it simply dereferences the JSON message for key “code”. Thus, the contents of the JSON messages used for various tasks are displayed in this section.

3.2.1. Leecher Messages

Figure 7 shows the message sent from the leecher to the seeder for registration. It contains the code for registering, the client type and the peer id, a random number generated by the leecher.

```
# Leecher Register request message
message = {
    "code": "register_request",
    "client_type": "leecher",
    "peer_id": user_id  # Unique ID for this leecher
}
```

Figure 7: Message sent by leecher to tracker for registration

Figure 8 shows the message sent by the leecher to the tracker to receive an array of seeders to request a certain file from. It contains the code, the name of the file desired and the client type. Again, the relative order of these fields in the data structure is irrelevant in this context.

```
# Leecher request file message
message = {
    "code": "FR",
    "filename": filename,
    "client_type": "leecher",

}
```

Figure 8: Message used by leecher to request file from network

Figure 8 shows the message sent by the leecher to the tracker after a download to indicate that the leechers that send it file chunks are now finished, and free to send chunks to other peers.

```
# Leecher Free Seeders Request
message = {
    "code": "free_seeders",
    "seeders": seeders,
    "client_type": "leecher",
    "checksum": state

}
```

Figure 8: Message used by leecher to indicate to tracker that the seeders that provided a file to it are now finished

3.2.2. Seeder Messages

Figure 9 shows the message sent by the seeder to the tracker to register with the tracker. The information included is the code, the client type, an array of the files the seeder is willing to share with the network and the peer id - a random number generated by the seeder.

```
# Seeder register request
message = {
    "code": "register_request",
    "client_type": "seeder",
    "available_files": files,
    "peer_id": user_id # Unique ID for this seeder
}
```

Figure 9: Message used by seeder to register with tracker

Figure 10 shows the message sent by the seeder to update the tracker about new or deleted files.

```
# Seeder update request
message = {
    "code": "update_request",
    "client_type": "seeder",
    "available_files": files,
    "peer_id": "user:"+str(user_id)# Unique ID for this seeder
}
```

Figure 10: Message used by seeder to update tracker about new or deleted files

3.2.3. Tracker Messages

Figure 11 shows the message sent by the tracker to a leecher or a seeder that has been successfully registered.

```
# Tracker registration success response message
response_message = {
    "code": "request_success"
}
```

Figure 11: Message sent by tracker to indicate a successful registration

Figure 12 shows the message sent by the tracker to a seeder that has successfully updated its record of the files the seeder is willing to share.

```
# Tracker update success message
response_message = {
    "code": "update_success"
}
```

Figure 12: Message sent by tracker to seeder for a successful update of available files

```
# Tracker file request response
response_message = {
    "code": "available_seeders",
    "file_seeders": file_seeders,
    "size": size
}
```

Figure 13: Message sent by tracker to leecher of available seeder IP addresses, port numbers and file size

```
# Tracker download complete message
response_message = {
        "code": "**Download Complete**",
    }
```

Figure 14: Message used by tracker to indicate a successful download

3. Screenshots

Figure 15 shows screenshots of all the features discussed in section 1.

6) 'somefile.txt' is entered by user into leecher terminal

user input

IP address and port numbers of free seeders with 'somefile.txt'

```
Enter a file you want:
somefile.txt
available_seeders
[['127.0.0.1', 59227], ['127.0.0.1', 43901]]
file size:  2000
checksum:  c50d51fdeb9b2fe5c804660f7f43a486
[+] Received 1000 bytes from 127.0.0.1:59227
checksum:  c50d51fdeb9b2fe5c804660f7f43a486
[+] Received 1000 bytes from 127.0.0.1:43901
[+] Appending received_chunk0.dat
[+] Appending received_chunk1.dat
Comparing:  c50d51fdeb9b2fe5c804660f7f43a486 c50d51fdeb9b2fe5c804660f7f43a486
Telling Server to free Seeders:
[['127.0.0.1', 59227], ['127.0.0.1', 43901]]
**Download Success**
**Download Success**
If not already, would you like to become a Leecher? (Y/N):
Y
Enter the directory path of the files you want to share: /home/shaw/Documents/Assignment_Code_Checksum
Enter a file you want:
```

checksum of expected file

comparing checksum of downloaded file with expected checksum

free seeders

option to switch to seeder

In this case, user decides to switch to seeder

7) Tracker shows the process of noting that seeders are busy, and then freeing the seeders

```
Current Busy Seeders:
[[('127.0.0.1', 59227), ('127.0.0.1', 43901)]]
Busy Seeders after download:
[]
Download Success:  True
Updater:
Updated Seeder: user:414 with data: {'client_type': 'seeder', 'available_files': [['test_hash.py', 206], ['somefile.txt', 2000],
 ['Tracker.py', 8848], ['downloadsomefile.txt', 2000], ['Client.py', 651], ['Leecher.py', 5272], ['Seeder.py', 5272]], 'IP': '12
7.0.0.1', 'Port': 59227}

Registered Seeder: user:673 with data: {'client_type': 'seeder', 'available_files': [['test_hash.py', 206], ['somefile.txt', 200
0], ['Tracker.py', 8848], ['downloadsomefile.txt', 2000], ['Client.py', 651], ['Leecher.py', 5272], ['Seeder.py', 5272]], 'IP':
'127.0.0.1', 'Port': 52593}
```

8) After deleting a file from the directory that all of the processes share, the tracker updates their information

```
Updater:
Updated Seeder: user:430 with data: {'client_type': 'seeder', 'available_files': [['test_hash.py', 206], ['somefile.txt', 2000],
 ['Tracker.py', 8848], ['Client.py', 651], ['Leecher.py', 5272], ['Seeder.py', 5272]], 'IP': '127.0.0.1', 'Port': 43901}

Updater:
Updated Seeder: user:414 with data: {'client_type': 'seeder', 'available_files': [['test_hash.py', 206], ['somefile.txt', 2000],
 ['Tracker.py', 8848], ['Client.py', 651], ['Leecher.py', 5272], ['Seeder.py', 5272]], 'IP': '127.0.0.1', 'Port': 59227}

Updater:
Updated Seeder: user:673 with data: {'client_type': 'seeder', 'available_files': [['test_hash.py', 206], ['somefile.txt', 2000],
 ['Tracker.py', 8848], ['Client.py', 651], ['Leecher.py', 5272], ['Seeder.py', 5272]], 'IP': '127.0.0.1', 'Port': 52593}
```

9) The same thing as in 6 above, but where the checksum fails because one seeder has a corrupted file

```
Enter a file you want:
somefile.txt
available_seeders
[['127.0.0.1', 59798], ['127.0.0.1', 51478]]
file size:  2000
checksum:  c50d51fdeb9b2fe5c804660f7f43a486
[+] Received 1000 bytes from 127.0.0.1:59798
checksum:  e9166be181f4e4a339e9f8e69c882b79
[+] Received 1000 bytes from 127.0.0.1:51478
[+] Appending received_chunk0.dat
[+] Appending received_chunk1.dat
Comparing:  e9166be181f4e4a339e9f8e69c882b79 6d44201085fd13ad01da07566c6e05e3
Something went wrong. Try again
Telling Server to free Seeders:
[['127.0.0.1', 59798], ['127.0.0.1', 51478]]
**Something went wrong. Try again.**
**Something went wrong. Try again.**
If not already, would you like to become a Leecher? (Y/N):
```

Figure 15: Screenshots of all the features discussed in section 1